

UNIVERSITÉ DE SHERBROOKE
Faculté de génie
Département de génie électrique et génie informatique

RAPPORT D'APP

Introduction à la programmation en robotique
APP 2

Présenté à
Eugène Morin et Domingo Palao Munoz

Présenté par
Vincent Duchesne – ducv1824
Simon Lacroix – lacs7291

Sherbrooke – 7 octobre 2026

TABLE DES MATIÈRES

1.	Introduction	1
2.	Développement	1
2.1	Identification du problème	1
2.2	Procédure de résolution	1
2.3	Mise en œuvre du programme	3
2.4	Analyse du résultat	4
3.	Conclusion	5
4.	Références	5

1. INTRODUCTION

La technologie lidar occupe une importance croissante dans le développement de nouvelles solutions automatisées, offrant une façon rapide, précise et efficace de détecter son environnement. C'est le concept à la base de l'APP 2, qui présente une problématique dont la solution repose sur le développement d'un algorithme de filtrage de données lidar [1]. Ce document présente la façon dont l'équipe a résolu celle-ci, en décrivant l'identification du problème, la procédure de résolution, la mise en œuvre du programme et l'analyse du résultat.

2. DÉVELOPPEMENT

2.1 IDENTIFICATION DU PROBLÈME

Cette problématique comporte plusieurs éléments à résoudre. Tout d'abord, les données provenant du lidar doivent être extraites du fichier texte et placées dans une liste afin de pouvoir être traitées par un filtre choisi par l'utilisateur parmi différents filtres, soit le filtre minimum-maximum, le filtre de moyenne mobile et le filtre de médiane glissante. L'analyse des fichiers d'entrée montre que les données sont fournies dans un fichier texte avec un chiffre par ligne. Pour le minimum-maximum, les bornes sont de 0,5, afin d'éliminer les points bloqués par le châssis, et de 15, afin d'éviter les données en dehors de la portée du lidar. Au niveau de la sortie, les données filtrées doivent également conserver le même format que les données d'entrée, soit une valeur par ligne avec une décimale. Finalement, des statistiques doivent être calculées à partir des données valides obtenues après le filtrage. Les valeurs égales à -1, qui représentent les données invalides, doivent être exclues des calculs. Les statistiques à produire sont les valeurs minimale, maximale, médiane et moyenne, ainsi que le nombre de points d'entrée et le nombre de points valides.

2.2 PROCÉDURE DE RÉOLUTION

Les trois algorithmes de traitement des données utilisent une boucle afin d'itérer sur chaque valeur de la liste de points d'entrée. Ils travaillent tous sur une nouvelle liste qui est retournée à la fin de leur exécution, pour éviter de modifier la liste originale passée en paramètre.

Pour le filtre minimum-maximum, chaque valeur est comparée à une limite maximale et une minimale, passées en paramètre et qui valent par défaut respectivement 0,5 et 15. Une valeur inférieure au minimum est remplacée par -1, et une supérieure au maximum est remplacée par le maximum. Si elle se trouve dans la plage valide, elle est conservée telle quelle. La figure 1 présente le diagramme d'activité pour cet algorithme.

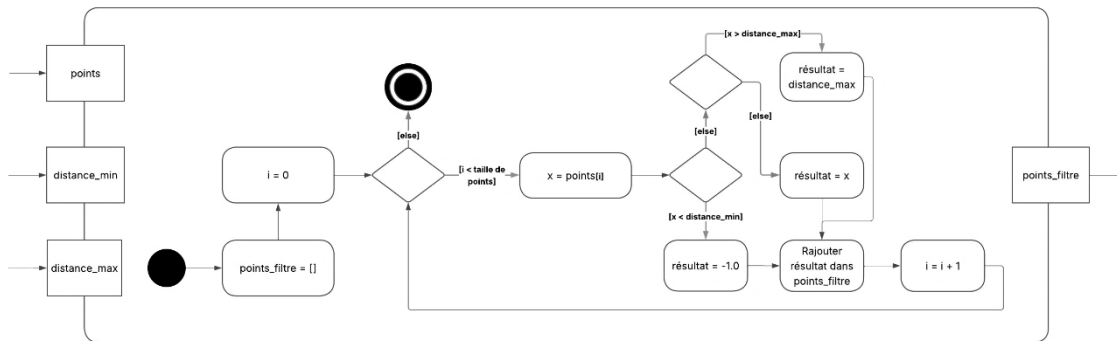


Figure 1 - Diagramme d'activité de l'algorithme minimum-maximum

Pour le filtre moyenne mobile, une itération sur les index des éléments de la liste de points d'entrée permet d'accéder facilement aux voisins de chaque élément, en plus de vérifier si l'élément est une extrémité de la liste. Ainsi, la variable *somme* contient la somme des éléments à moyenner alors que *nbElements* contient le nombre d'éléments. Les deux éléments aux extrémités prennent seulement en compte l'élément lui-même et son seul voisin, alors que les autres situés au milieu de la liste utilisent l'élément et ses deux voisins. Une division de *somme* par *nbElements* permet d'obtenir la moyenne avec les éléments pertinents. La figure 2 présente le diagramme d'activité pour cet algorithme.

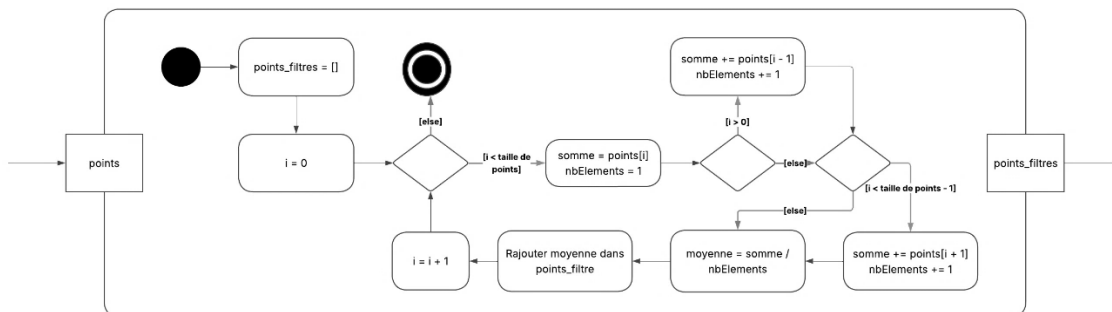


Figure 2 - Diagramme d'activité de l'algorithme de moyenne mobile

Le même principe d'itération sur les index est appliqué pour le filtre de médiane glissante. Les valeurs situées aux extrémités de la liste sont conservées telles quelles. Pour les autres, la valeur actuelle ainsi que ses deux valeurs adjacentes sont mises dans une liste qui est ensuite triée par ordre croissant. La médiane correspond donc à la valeur au milieu de la liste (index 2). La figure 3 présente le diagramme d'activité pour cet algorithme.

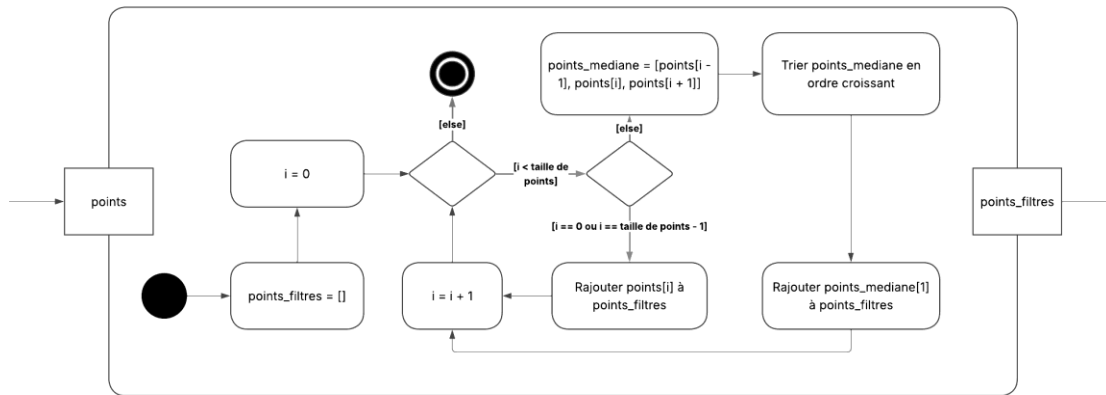


Figure 3 - Diagramme d'activité de l'algorithme de médiane glissante

2.3 MISE EN ŒUVRE DU PROGRAMME

Le code développé est organisé en 6 modules pour bien séparer les responsabilités.

Le module *banc_lidar* est le point d'entrée du programme lors d'une utilisation normale, qui coordonne l'échange de données entre les différents modules et qui s'occupe de la boucle d'action principal. Il permet à l'utilisateur de recommencer l'exécution du programme avec d'autres paramètres une fois l'opération de filtrage terminée. Ce module gère les arguments passés par ligne de commande, au cas où l'utilisateur passe le fichier d'entrée de cette façon; sinon, le programme demande à l'utilisateur le fichier à utiliser.

Le module *interaction_usager* regroupe les fonctions qui demandent à l'utilisateur d'entrer une valeur, que ce soit pour les noms des fichiers d'entrée et de sortie, pour le choix de l'algorithme à utiliser ou pour demander l'action qu'il souhaite faire une fois le filtrage effectué. C'est aussi dans ce module qu'est gérée la validation des réponses de l'utilisateur, pour s'assurer que l'information que les fonctions retournent puisse être correctement utilisée dans le reste du programme. Notamment, il est validé que les fichiers d'entrée se trouvent dans le répertoire *entree* et les fichiers de sortie sont placés dans le répertoire *sortie* avec l'extension *.txt*.

Le module *entree_sortie* s'occupe de la lecture de données du fichier d'entrée et de l'écriture des données de sortie filtrées.

Le module *filtrage* contient les implémentations des trois algorithmes de filtrage de la problématique, qui prennent une liste de données en entrée et retournent une liste filtrée.

Le module *analyse* gère le calcul et l'affichage des statistiques, avec une implémentation d'un algorithme de tri par insertion [2] nécessaire pour le calcul de la médiane des données.

Finalement, le module *test_filtrage* offre un point d'entrée alternatif servant seulement à valider le fonctionnement des algorithmes de filtrage. Les tests implémentés couvrent les cas limites des différents filtres afin de s'assurer que les algorithmes fonctionnent correctement avec différentes listes de données. Pour chacun des filtres, les tests utilisent des listes vides, des listes avec un élément et des listes avec plusieurs éléments. Les tests utilisent aussi des valeurs variées comme le 0, les négatifs et les chiffres à virgule. Un cas spécifique est ajouté pour le filtre de médiane glissante, soit une liste comportant deux chiffres identiques.

2.4 ANALYSE DU RÉSULTAT

La solution présentée à la problématique comporte plusieurs forces. D'abord, le code est organisé de façon structurée avec une division en trois dossiers : *src* pour les scripts Python, *entree* pour les fichiers d'entrée et *sortie* pour les fichiers de sortie. Cela rend l'utilisation du programme plus intuitive en évitant une confusion quant à l'emplacement des fichiers. De plus, tout le code est divisé en modules composés de fonctions à responsabilité bien définie, sans aucune variable globale, ce qui facilite la réutilisation de code dans d'autres parties du projet. Un autre point fort est la gestion d'erreurs. Le nom du fichier d'entrée et le numéro de l'algorithme choisi entrés par l'utilisateur sont validés pour s'assurer qu'ils sont corrects (que le fichier d'entrée existe et que le choix d'algorithme soit compris entre 1 et 3). Le répertoire *sortie* est aussi créé s'il n'existe pas déjà au moment d'écrire le fichier de sortie pour éviter des erreurs d'emplacement de fichier, et l'extension du fichier est rajoutée ou remplacée pour être *.txt*. Finalement, le programme est exécuté selon une boucle gérée par l'utilisateur, ce qui facilite le filtrage consécutif de données. Cela permet d'aisément appliquer un nouveau filtre sur le même fichier d'entrée, sur le dernier fichier de sortie créé ou sur un nouveau fichier d'entrée.

Cependant, la solution développée présente aussi certains points à améliorer. Notamment, les données du fichier d'entrée doivent être dans le format attendu, soit un chiffre individuel par ligne. Un format différent mènerait à une exception dans le code, puisque la fonction *float()* est utilisée pour chaque ligne. De plus, il est difficile de savoir si les algorithmes développés sont bien adaptés pour le système embarqué du robot, puisqu'il est possible que certains ajustements soient nécessaires avant d'implémenter le programme. Par exemple, le robot nécessitera probablement une structure de décision qui pourra choisir l'ordre dans lequel les filtres sont appliqués pour obtenir un résultat optimal pour son algorithme de positionnement. Des tests d'implémentation sont donc certainement nécessaires, puisque cette solution a été développée seulement à partir de fichiers de données tests qui ne sont pas nécessairement représentatifs de tous les cas possibles pour le vrai robot.

3. CONCLUSION

Pour conclure, à travers la démonstration de l'identification du problème, de la procédure de résolution, de la mise en œuvre du programme et de l'analyse du résultat, ce rapport démontre le processus de développement de la solution à la problématique de l'APP 2. Le programme réalisé répond à toutes les exigences principales en permettant de lire un fichier texte, d'y passer un certain filtre et de renvoyer les données filtrées dans le même format qu'à l'arrivée tout en montrant les statistiques des données valides.

4. RÉFÉRENCES

- [1] F. Ferland et E. Morin, «Session S2,» 2026. [En ligne]. Available: https://session.genie.usherbrooke.ca/a2026/s1gro/app_2_sntetm_gro/documents/gro120-a26-guide_études.pdf. [Accès le 5 Octobre 2026].
- [2] Wikipédia, «Tri par insertion,» 31 Août 2026. [En ligne]. Available: https://fr.wikipedia.org/wiki/Tri_par_insertion. [Accès le 6 Octobre 2026].